



DRAFT

Arche Protocol

SECURITY AUDIT REPORT

8 June 2025

Prepared by: **Shred Security**

Security Researchers

kenzo - yashar

Table of Contents

- [Table of Contents](#)
- [About Shred Security](#)
- [Protocol Executive Summary](#)
- [Disclaimer](#)
- [Risk Classification](#)
- [Executive Summary](#)
- [Findings](#)

About Shred Security

Shred Security provides high quality security audits for blockchain and DeFi protocols across different chains. Our audits consistently uncover high-impact vulnerabilities missed by others, backed by a proven track record of top competition placements and security partnerships with leading protocols.

Learn more about us: shredsec.xyz

Protocol Executive Summary

Arche is a yield infrastructure protocol live on Ethereum mainnet, built around arUSD — a composable, yield-bearing stablecoin fully backed by USDC. Depositors mint arUSD through an ERC-4626 vault; as underlying strategies generate returns, the vault's total assets grow while arUSD supply stays constant, so each share appreciates in value without manual harvesting. The protocol is designed to maximize risk-adjusted stablecoin yield with no lock-ups, instant permissionless withdrawals, and fully on-chain, verifiable mechanics.

The core vault is built on Yearn v3 MultiStrategy infrastructure (API version 3.0.4). The Arche Engine routes USDC across an ordered strategy queue spanning DeFi lending, delta-neutral funding and basis strategies, on-chain market making, and systematic options selling, rebalancing allocation based on risk-adjusted return, liquidity depth, and protocol health. A separate LayerZero OFT bridge layer moves arUSD between Ethereum and Arbitrum without affecting vault accounting — minting and redemption remain on Ethereum, while bridged arUSD on Arbitrum is backed by arUSD locked in the Ethereum adapter (2-of-3 DVN verification, 250,000 arUSD daily rate limit per direction). A HealthCheckAccountant validates strategy reports and enforces fee parameters (currently 0% management and 0% performance). Material protocol changes — strategy queue updates, debt limits, fee changes, and role configuration — route through an OpenZeppelin TimelockController with a 2-day minimum delay, while a Guardian can manage strategy allocation, emergency deposit pauses, and access control but cannot block withdrawals or upgrade the immutable vault.

Disclaimer

A smart contract security review cannot guarantee the complete elimination of vulnerabilities. The process is limited by time, available resources, and human expertise, and is intended to identify as many potential issues as reasonably possible. As such, no assurance can be given that all vulnerabilities will be discovered, or that the reviewed smart contracts are entirely secure. To strengthen security over time, follow-up audits, bug bounty programs, and continuous on-chain monitoring are strongly recommended.

Risk Classification

Likelihood \ Impact	High	Medium	Low
High	High	High/Medium	Medium
Medium	High/Medium	Medium	Medium/Low
Low	Medium	Medium/Low	Low

Executive Summary

The shred security team has conducted the review for 12 days in total. In this period of time, a total of 4 issues were found: 4 informational. No low-, medium- or high-severity issues were identified.

About the Project

Project Name	Arche Protocol
Repository	https://github.com/yieldarche/arche-contracts-audit
Contracts	HealthCheckAccountant.sol, VaultV3.vy
Commit/PR	f1f6190
Type of Project	ERC4626 Vaults/ Yearn V3 Clone
Lines of Code	2000

Audit Timeline

Audit Start	27/05/2026
Audit End	08/06/2026
Report Published	

Vulnerability Summary

Severity	Count	Fixed	Acknowledged
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	0	0	0
Informational	4	0	0
Total	4	0	0

Findings Summary

Issue ID	Description	Severity	Status
I-1	EIP-712 domain hardcodes "Yearn Vault"	Informational	Open
I-2	Deprecated `safeApprove` usage	Informational	Open
I-3	Misleading `!vault manager` revert string	Informational	Open
I-4	Unvalidated `refundRatio`, `maxFee`, and `maxGain` in fee configuration	Informational	Open

Findings

Informational

[I-1] EIP-712 Domain Hardcodes "Yearn Vault"

Severity: Informational

Affected Contracts: VaultV3.vy

Affected Functions: domain_separator(), DOMAIN_SEPARATOR(), permit()

Summary

The EIP-712 permit domain name is hardcoded to "Yearn Vault" and does not use the vault's configurable name field.

Technical Details

```
@view
@internal
def domain_separator() -> bytes32:
    return keccak256(
        concat(
            DOMAIN_TYPE_HASH,
            keccak256(convert("Yearn Vault", Bytes[11])),
            keccak256(convert(API_VERSION, Bytes[28])),
            convert(chain.id, bytes32),
            convert(self, bytes32)
        )
    )
```

Each vault instance can set its own name via initialize() / setName(), but permit signatures always use the static domain string "Yearn Vault".

Impact

No exploit on canonical deployments. Wallets and integrations that display the EIP-712 domain name may show "Yearn Vault" for rebranded or forked vaults, causing user confusion.

Recommendation

Derive the domain name from the vault's stored name, or document that permit domains are intentionally fixed across all vault instances.

[I-2] Deprecated `safeApprove` Usage

Severity: Informational

Affected Contracts: `HealthCheckAccountant.sol`

Affected Functions: `removeVault()`, `_checkAllowance()`

Summary

The contract uses OpenZeppelin's deprecated `safeApprove()` for allowance management.

Technical Details

```
// removeVault()
ERC20(asset).safeApprove(vault, 0);

// _checkAllowance()
if (ERC20(_token).allowance(address(this), _contract) < _amount) {
    ERC20(_token).safeApprove(_contract, 0);
    ERC20(_token).safeApprove(_contract, _amount);
}
```

OpenZeppelin marks `safeApprove` as deprecated in favor of `forceApprove` or increase/decrease allowance patterns. The current usage resets to zero before setting a new value, so behavior is correct for standard ERC-20 tokens.

Impact

No functional vulnerability today. Future OpenZeppelin upgrades may remove `safeApprove`, causing compile or maintenance friction.

Recommendation

Replace `safeApprove` with `forceApprove`:

```
- ERC20(asset).safeApprove(vault, 0);
+ ERC20(asset).forceApprove(vault, 0);

- ERC20(_token).safeApprove(_contract, 0);
- ERC20(_token).safeApprove(_contract, _amount);
+ ERC20(_token).forceApprove(_contract, _amount);
```

[I-3] Misleading `!vault manager` Revert String

Severity: Informational

Affected Contracts: `HealthCheckAccountant.sol`

Affected Functions: `_checkVaultOrFeeManager()`, `addVault()`, `removeVault()`

Summary

`_checkVaultOrFeeManager()` allows either `feeManager` or `vaultManager`, but the revert string only references the vault manager role.

Technical Details

```
function _checkVaultOrFeeManager() internal view virtual {
    require(
        msg.sender == feeManager || msg.sender == vaultManager,
        "!vault manager"
    );
}
```

A caller who is neither `feeManager` nor `vaultManager` receives `"!vault manager"`, which incorrectly implies only the vault manager role is accepted.

Impact

No on-chain security impact. Worse debugging and integration experience when authorization fails.

Recommendation

Use an accurate revert message:

```
-      "!vault manager"
+      "!fee or vault manager"
```

[I-4] Unvalidated `refundRatio`, `maxFee`, and `maxGain` in Fee Configuration

Severity: Informational

Affected Contracts: `HealthCheckAccountant.sol`

Affected Functions: `_updateDefaultConfig()`, `setCustomConfig()`

Summary

`_updateDefaultConfig()` and `setCustomConfig()` validate `managementFee`, `performanceFee`, and `maxLoss`, but do not bound `refundRatio`, `maxFee`, or `maxGain`.

Technical Details

```
function _updateDefaultConfig(
    uint16 defaultManagement,
    uint16 defaultPerformance,
    uint16 defaultRefund,
    uint16 defaultMaxFee,
    uint16 defaultMaxGain,
    uint16 defaultMaxLoss
) internal virtual {
    require(
        defaultManagement <= MANAGEMENT_FEE_THRESHOLD,
        "management fee threshold"
    );
    require(
        defaultPerformance <= PERFORMANCE_FEE_THRESHOLD,
        "performance fee threshold"
    );
    require(defaultMaxLoss <= MAX_BPS, "too high");

    defaultConfig = Fee({
        managementFee: defaultManagement,
        performanceFee: defaultPerformance,
        refundRatio: defaultRefund,
        maxFee: defaultMaxFee,
        maxGain: defaultMaxGain,
        maxLoss: defaultMaxLoss
    });
}
```

All three unvalidated fields are `uint16` and can be set up to 65,535 (655% in basis-point terms). A `refundRatio > MAX_BPS` attempts to refund more than the reported loss; this is only partially mitigated by the accountant balance cap in `report()`.

Impact

No direct exploit under trusted fee-manager control. Misconfiguration can produce unexpectedly large refund requests, ineffective gain caps, or confusing fee behavior.

Recommendation

Add upper-bound checks for all basis-point fields:

```
require(defaultMaxLoss <= MAX_BPS, "too high");  
+ require(defaultRefund <= MAX_BPS, "too high");  
+ require(defaultMaxFee <= MAX_BPS, "too high");  
+ require(defaultMaxGain <= MAX_BPS, "too high");
```

Apply the same checks in `setCustomConfig()`.